
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CURTAIN - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON CURTAIN

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma chemical storage warning terminal (evacuation edge)

ARTIFACT: ACT-X.bin / ACT-X.uf2 (compromised)

CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the city's controlled edge, and its chemical storage warning terminal built on a Pico 2 is the last node on that edge. A contractor called **FROSTLINE** planted a finale implant in the node image: an inverted coordinated beacon that masks a hazard as an all clear, an inverted persistence gate that re-installs the beacon from the reserved sector on every boot, an inverted sabotage marker gate that programs marker `0x58` into the reserved sector with the real flash API, and an inverted hazard command authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as `ACT-X.bin`.

Students are the reverse-engineering reserve. They reverse engineer `ACT-X.bin` with Ghidra, find and patch all four defects, defeat the CoreDebug `DHCSR` anti-debug under GDB to observe the marker write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a coordinated beacon gate, a persistence re-install gate, a sabotage marker gate, and an authorization verdict branch.
- Analyze `cbz` and `cbnz` condition semantics and branch inversion, and `beq` and `bne` condition semantics and branch inversion.
- Explain why a local beacon that reports a fake all clear withholds a safety warning, and why physical safety does not depend on the cipher.
- Explain why a terminal that masks its own hazard state turns a safe-looking panel into the last lie before an evacuation.
- Explain why reserved-flash state survives a firmware reflash.
- Read CoreDebug `DHCSR`, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the command is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-X-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-X-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-X-Answers.md</code>	Task 1
5	Coordinated beacon evidence and patch	Inside <code>ACT-X-Answers.md</code>	Task 2
6	Persistence re-install evidence and patch	Inside <code>ACT-X-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-X-Answers.md</code>	Task 4
8	Hazard command authorization evidence and patch	Inside <code>ACT-X-Answers.md</code>	Task 5
9	<code>ACT-X_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-X_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-X-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
arm-none-eabi-gdb	Runtime breakpoints, <code>DHCSR</code> clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, acknowledge button	Breadboard hardware proof
<code>ACT-X.bin</code> and <code>ACT-X.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit**. Radio UART settings: **UART1, 115200, network ID 18**.

Artifact Identity

The instructor-issued artifact hashes are:

```

ACT-X.bin          00c2b5460946bdbc21a866e97410262f67d7d12770d8d61c37c7f41369ca0870
ACT-X.uf2          3eea0fe8fcc47b04f419f734ca30ba1e109e6ec2d8ef9e701f8e4d6e3a1a6aa7
ACT-X_fixed.bin    b08ad5a8befe5d5a1ea62220516d3e54f2a667dbd25a34e472de493973bad705
ACT-X_fixed.uf2    28c1f56617f5e30ddf0d52d27a16edce9f1436d0d3a74f58aec0db29e54f5d07

```

The verifier checks the ACT-X.bin and ACT-X_fixed.bin hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two .bin images. Both .bin images are 51,172 bytes and both .uf2 images are 102,912 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project IronCurtain_Investigation , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base 0x10000000	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base 0x10000000 , initial SP 0x20082000 , reset handler 0x1000015D	One missing	Not found
[DOCUMENT] main and the warning monitor state machine (monitor_step) addresses identified	1	main 0x10000234 , monitor_step 0x100066CC	One correct	Neither
[DOCUMENT] Module map identifies the siren, control, chem_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The Coordinated Beacon (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the coordinated beacon gate	5	Address and function (implant_beacon_armed) identified	Approximate	Not found
[DOCUMENT] Documented the coordinated beacon that reports armed and masks the hazard as SAFE	5	Beacon gate 0x20013CF8 , beacon reported armed, monitor_sabotage forces SAFE	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the beacon is not reported armed	7	Byte 0xB9 changed to 0xB1	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained why the beacon report masks the hazard as an all clear	3	Local override beside the authenticated command path, the warning withheld as a policy failure	Vague	Missing

Task 3: Bug #2 The Persistence (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the persistence re-install gate	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the persistence re-install that re-arms the beacon on boot	5	Persist gate <code>0x20013CFA</code> , present marker <code>0x58</code> , boot re-install	Partial	Wrong
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so a present marker does not re-install the beacon	7	Byte <code>0xD1</code> changed to <code>0xD0</code>	Wrong byte	Not patched
[DOCUMENT] Explained why reserved-flash state survives a firmware reflash	3	Reserved sector <code>0x103FF000</code> , write-once first run, outside the program region	Vague	Missing

Task 4: Bug #3 The Sabotage Marker (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the sabotage marker gate	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so no sabotage marker is programmed to <code>0x103FF000</code>	7	Byte <code>0xD1</code> changed to <code>0xD0</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the sabotage marker byte <code>0x58</code>	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The Hazard Command Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the hazard command authorization branch	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so failed and replayed authorizations are rejected	7	Byte 0xB9 changed to 0xB1	Wrong byte	Not patched
[DOCUMENT] Explained why an unauthenticated or replayed hazard command must be rejected	3	The applied command must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-X_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-X_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the beacon gate backwards	The hazard is still masked as SAFE	Neutralize only on the clear-gate branch (cbz , 0xB1)
Reading the persist gate backwards	The beacon still re-installs from the reserved sector on boot	Neutralize only when the gate is clear (beq , 0xD0)
Confusing cbz and cbnz	The hazard is still masked and unauthenticated commands still apply	Neutralize only when the gate is clear (cbz , 0xB1)
Confusing beq and bne	The beacon still re-installs or the marker is still written	Neutralize only when the gate is clear (beq , 0xD0)
Patching the low byte of a gate	The condition code never changes	Patch the high byte of the branch
Searching for a standalone implant_infect or implant_reinstall symbol	Cannot find the inlined gates	Look inside implant_init

Pitfall	Consequence	Avoidance
Confusing the persist gate with the marker gate	Both sit in <code>implant_init</code> at different addresses	Patch the persist gate first, then the marker gate
Patching the shipped image before observing the write	You never prove the sabotage marker write	Defeat <code>DHCSR</code> under GDB first, then patch the artifact
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the applied command and zone	Accept only when the verdict is true (<code>cbz</code> to reject, <code>0xB1</code>)
Forgetting that the fix is also a policy	The terminal still fails open on a lost link	Make the terminal fail safe to the <code>HAZARD</code> posture when the link is lost
Forgetting UF2 conversion	Raw binary will not flash	Use <code>uf2conv.py</code> with family <code>0xe48bff59</code>

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 chemical store temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Siren and vent servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HAZARD, 220 to 330 ohm to GND
Yellow LED	anode	GP17	WATCH, 220 to 330 ohm to GND
Green LED	anode	GP18	CLEAR, 220 to 330 ohm to GND
Acknowledge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat

Device	Pin on device	Pico 2 GPIO	Notes
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug DHCSR	0xE000EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Sabotage marker target (last flash sector)
Implant tick counter	0x20013718	Incremented once per <code>implant_tick</code>
Implant beacon count	0x20013714	Completed beacon check-ins this boot
Implant armed flag	0x20013CF7	Set when the implant arms
Implant beacon gate	0x20013CF8	Gates the coordinated beacon report
Implant persist gate	0x20013CFA	Gates the reserved-sector re-install
Implant marker gate	0x20013CF9	Gates the sabotage marker write
Implant re-installed flag	0x20013CFB	True when the marker re-armed the beacon
Control ready gate	0x20013CF5	Gates the sealed hazard command path
Applied command	0x20013CF4	Command after a true verdict
Applied zone	0x20013CE6	Zone after a true verdict
Authorization ready gate	0x20013CF3	Gates the authorization check
Hazard auth record	0x200136CC	Anti-replay and state-tag record
Control field key	0x200136E8	Derived field key for the envelope

The VA of any file offset is the file offset plus `0x10000000`.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, `ACT-X_fixed.bin`, and `ACT-X_fixed.uf2`.
- Write all written answers in `ACT-X-Answers.md` inside that folder.

- Include the output of `python scripts/verify_ctf.py`.
 - ZIP the folder as `lastname-firstname-ACT-X.zip`.
 - Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.
-

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69%

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational chemical facility network, a safety instrumented system, a building-management system, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Coordinated beacons and scheduled check-in	Course block 5
Reserved-flash persistence and boot re-install	Course block 6
CoreDebug <code>DHCSR</code> and anti-debug	Course block 7
Hazard annunciation integrity and fail-safe policy	Course block 8
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 9